

LogPress 평가 프로토콜 재설계 및 구현 현황

기존 line 생존 판정의 문제점 → 교수님 피드백 이후 재정의 → 현재 구현 상태

2026.09.28 팀 회의용

오늘의 핵심 결론

Metric은 바꾸지 않고, “압축 후 line 생존 판정 방식”을 바꾼다.

Cause/Evidence Line Retention은 유지 · token survival을 layer × KV head 단위로 line에 projection

기존 문제

token 1개만 남아도 line 생존
head union으로 보존율 과대평가

새 정의

slot별 token survival ratio → 평균 LineScore
고정 line budget에서 Top-K 선택

현재 작업

ObservedAttention부터 공통 evaluator로 리팩토링
개발: Qwen2.5-3B / 본 실험: Qwen2.5-7B

1. 기존 평가 설계의 문제점

왜 compression ratio를 바꿔도 line retention이 거의 안 변했는가

기존 판정 규칙

“어느 KV head에서든 token 하나만 남으면 line 생존”

- line에 token이 20개여도 1개만 남으면 KEEP
- 20개가 모두 남은 line과 동일하게 1점 처리
- compression ratio 0.5 / 0.7 / 0.9 차이가 line 수준에서 희석
- 초기 구현은 특정 layer를 보고 head token을 union하여 평가

결과적으로 생긴 문제

- 대부분의 line에 token 하나쯤 남아 retention이 비정상적으로 높아짐
- 압축 강도를 높여도 line retention 곡선이 거의 평평하게 나올 수 있음
- KV head 수가 많은 모델일수록 “어딘가에서 생존”할 기회가 늘어나는 편향 가능
- 모델 구조가 바뀌면 head 수 하드코딩 때문에 코드 재사용성이 떨어짐
- 결국 “얼마나 많이 살아남았는지” 정보가 line binary 판정에서 사라짐

핵심: metric 자체보다, KV token 결과를 line 생존으로 바꾸는 projection 규칙이 문제였다.

2. KV head별 생존 토큰 차이가 왜 문제인가

head union을 피해야 하는 이유

예시: 한 log line = 10 tokens

t0

t1

t2

t3

t4

t5

t6

t7

t8

t9

Head A

KEEP

KEEP

KEEP

KEEP

KEEP

KEEP

DROP

KEEP

DROP

KEEP

$8 / 10 = 0.8$

Head B

DROP

KEEP

DROP

DROP

KEEP

DROP

DROP

DROP

KEEP

DROP

$3 / 10 = 0.3$

기존 union 판정

둘 중 한 head에서만 살아도 token 생존 → line은 거의 항상 KEEP

새 slot-wise 관점

Head A=0.8, Head B=0.3처럼 독립 보존율을 유지한 뒤 평균

3. 교수님 미팅 이후 새로 정의한 평가 프로토콜

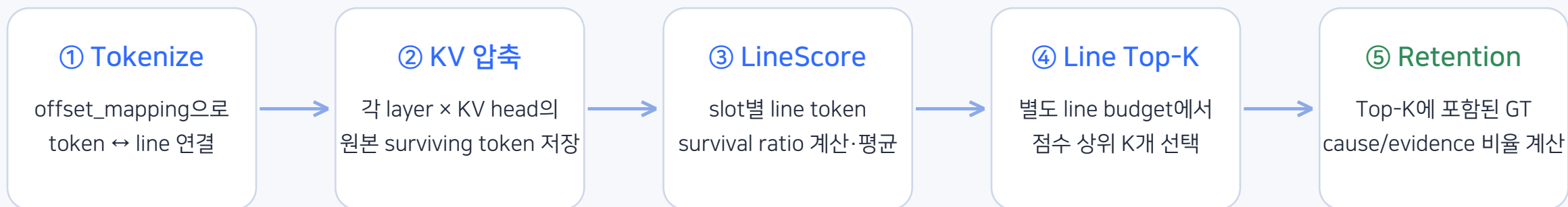
Metric은 고정, KV → Line Projection만 재설계

변경하지 않은 것

Cause/Evidence Line Retention = 살아남은 GT line / 전체 GT line

새로 정의한 것

“어떤 line이 살아남았는가”를 정하는 projection protocol



LineScore(line i) = 평균 over compressed layer × KV head [생존한 line i token 수 / line i 전체 token 수]

최종 보존율 = Top-K에 포함된 GT cause/evidence line 수 / 전체 GT cause/evidence line 수

KVDiagnosis에서는 slot-wise token retention 관점만 참고하고, line score·Top-K projection은 LogPress 평가 protocol로 구분

4. 현재 구현 상태와 다음 작업

ObservedAttention을 공통 evaluator의 첫 기준으로 리팩토링 중

현재 하고 있는 일

- ObservedAttention의 surviving token index 추적 코드는 이미 동작
- 기존 layer 0 + head union 평가를 제거하고 모든 layer × KV head로 LineScore 계산
- 개발: Qwen2.5-3B / 본 실험: Qwen2.5-7B

코드 설계 원칙

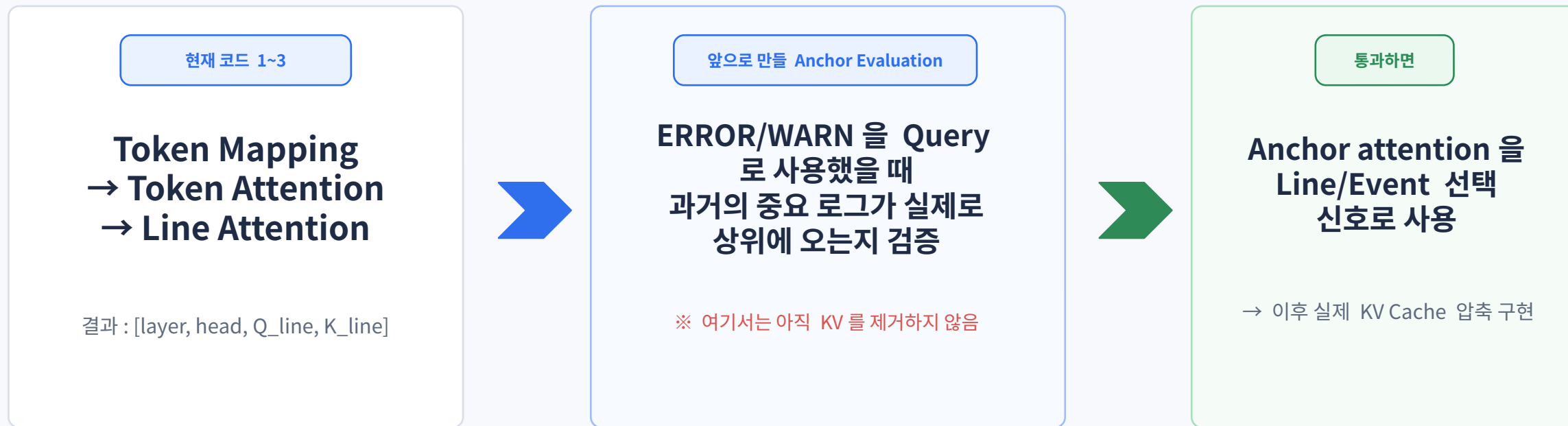
- KV head·layer 수·head_dim 등을 tensor/config에서 동적으로 읽기
- baseline은 [layer, kv_head, original token] survival 정보까지만 출력
- 공통 evaluator를 SnapKV / H2O / StreamingLLM에도 재사용

아직 남은 핵심 설계 이슈

- KV compression ratio 50%와 line Top-K 50%는 같은 budget이 아님 — token/KV-entry budget과 line-count budget을 분리해 정의해야 함
- line 길이가 서로 다르므로 동일 line 비율이 동일 KV memory 절감량을 의미하지 않음
- 현재 evaluator에서는 line_keep_ratio를 별도 파라미터로 두고, KV budget ↔ line budget 연결 규칙은 추가 결정
- Thunderbird anomaly는 RCA cause/evidence와 동일하지 않으므로 최종 GT 평가는 실제 cause/evidence 라벨이 확인된 데이터로 수행

Anchor Evaluation 은 무엇을 만들 계획인가 ?

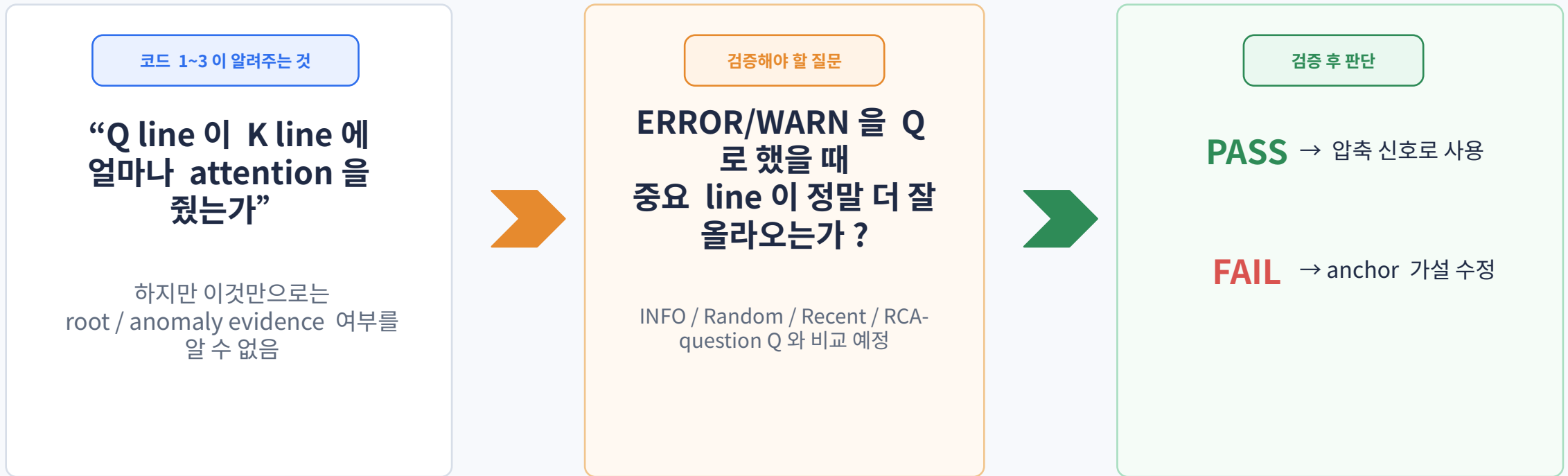
아직 완성된 압축 코드가 아니라 , ERROR/WARN anchor 가설을 먼저 검증하기 위한 사전 평가 단계



핵심 : “Attention 이 높다 = 중요하다” 를 전제로 쓰지 않고 , 먼저 실제 evidence 와 맞는지 검증한 뒤 압축에 사용한다 .

왜 Anchor Evaluation 이 필요한가 ?

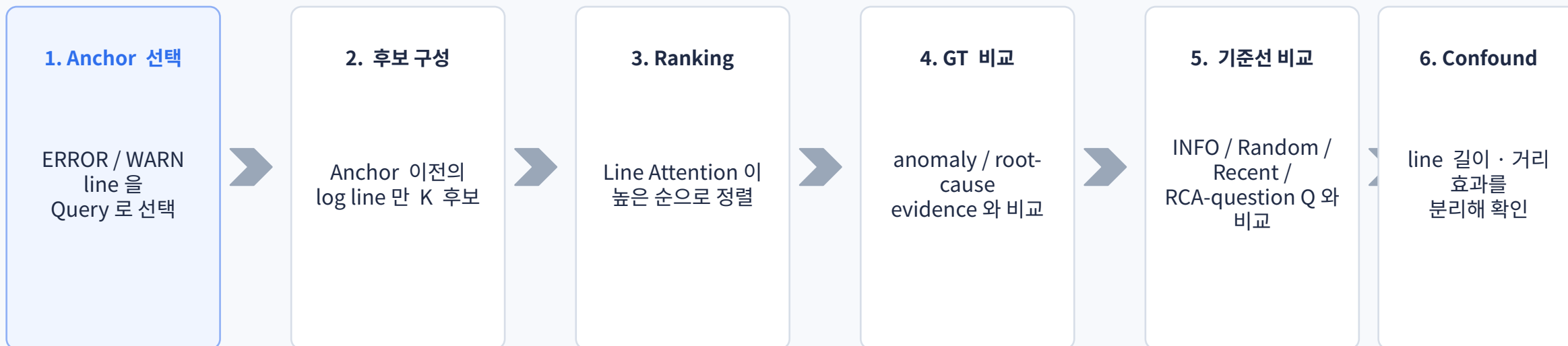
코드 1~3 은 line attention 을 계산해 주지만 , 그 값이 실제로 중요한 로그를 의미하는지는 아직 모름



즉 , Anchor 단계는 실제 압축 구현 전의 Go / No-Go 검증 단계이다 .

Anchor Evaluation 구현 계획

아직 구현 중인 단계이며, 아래 순서로 사전 검증 코드를 만들 계획



평가 계획

향후 ranking 성능은 Recall@K · MRR · AP 등의 retrieval metric 으로 볼 예정이며, 이는 PDF 의 1 층 본 평가와 별개인 “Anchor 사전검증용” 지표이다.

※ PDF 본 평가의 1 층은 압축 후 보존율 + marginal gain 중심

Anchor 코드 이후에 남는 구현

Anchor 가설이 통과하면 그때부터 실제 LogPress 압축기를 구현

5

Importance / Selector

Anchor attention 을
이용해
남길 line / event 를 결정

6

KV Compression

선택된 line 의 KV 는
유지하고
나머지 KV 를 제거

7

1 층 평가 · 정보 보존

보존율 · marginal gain
압축률별 비교 ·
ablation

8

2 층 평가 · 효율 / 비용

KV memory · TTFT
throughput · inference
time

9

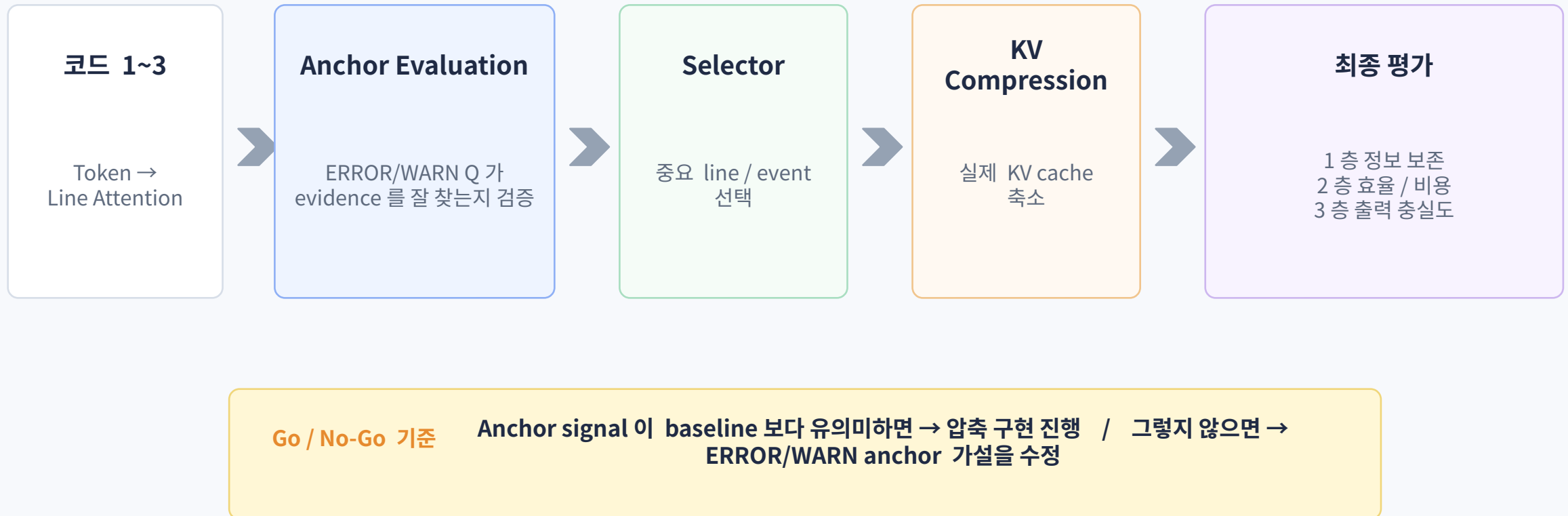
3 층 평가 · 출력 충실도

log-probability ·
perplexity
KL divergence

추가로 최소 RCA 평가 (K 지선다 / 원인 line 지목) 를 보조적으로 붙여 “보존한 정보가 실제 판단에도 유효한지” 확인할 계획.

전체 연구 흐름에서의 위치

현재는 “Line Attention 생성” 이후, 실제 압축 구현 전에 Anchor 가설을 확인하는 단계



LOGPRESS · 평가설계

압축을 어떻게 평가할 것인가

1·2·3 층 평가 — 무엇을 재고 · 어떻게 재고 · 어떤 검정으로

평가는 3 층 구조

각 층이 서로 다른 질문에 답한다. 오늘은 1 층은 간략히, 2,3 층 검정을 중심으로.

1

정보 보존

원인을 잘 남기나 (품질)

메트릭

- 보존율
- marginal gain

2

효율 / 비용

가볍고 빠르거나 (비용)

메트릭

- KV memory 절감
- TTFT · throughput
- inference time

3

출력 충실도

원본과 같게 판단하나

메트릭

- log-probability
- perplexity
- KL divergence

이번 발표 범위 1 층은 이미 정립 → 간략히. 2,3 층 검정이 무엇을 하는지를 설명한다.

1 층 — 정보 보존 (간략)

무엇을 재나

정답 원인 라인 중 , 압축 후 살아남은 비율 = 보존율

$$\text{보존율} = |G \cap C| / |G|$$

G = 정답 라인 전체 **C** = 압축기가 남긴 라인

핵심 지표 · marginal gain = 보존율 (LogPress) - 보존율 (레벨필터)

어떤 검정

McNemar

두 압축기가 같은 라인을 채점 → 짝진 승부 . 한쪽만 살린 라인만 비교 ($b > c$).

BH 보정

검정 60 번이라 우연 승리가 뜸 → 가짜 비율만 5% 로 눌러 진짜 차이를 살림 .

세부는 기존 평가설계 문서에 정립되어 있어 오늘은 요약만 .

2 층 — 효율 / 비용 : 지표가 무엇을 하나

무엇을 원인을 유지하면서 메모리 · 속도를 얼마나 줄이나 . 왜 실측 메모리 · 속도는 이진 승패가 아니라 측정값 → McNemar 안 씀 .

KV memory reduction

압축률별 KV 캐시가 얼마나 작아지나 . 같은 GPU 에서 더 긴 로그 수용 .

TTFT (Time To First Token)

첫 토큰까지 시간 . 긴 로그 읽는 (prefill) 시간이 여기 다 들어감 — 압축 효과 직격 .

throughput

초당 처리 토큰 수 . KV 가 작아지면 ↑ . 높을수록 좋음 .

inference time

입력 ~ 답 끝까지 전체 추론 시간 .

핵심 프레이밍 : $\text{retained evidence 단위당 절감} = \text{절감한 메모리 (또는 시간)} / \text{보존한 원인 라인 수}$

3 층 — 출력 충실도 : 원본과 같게 판단하나

출력 분포란

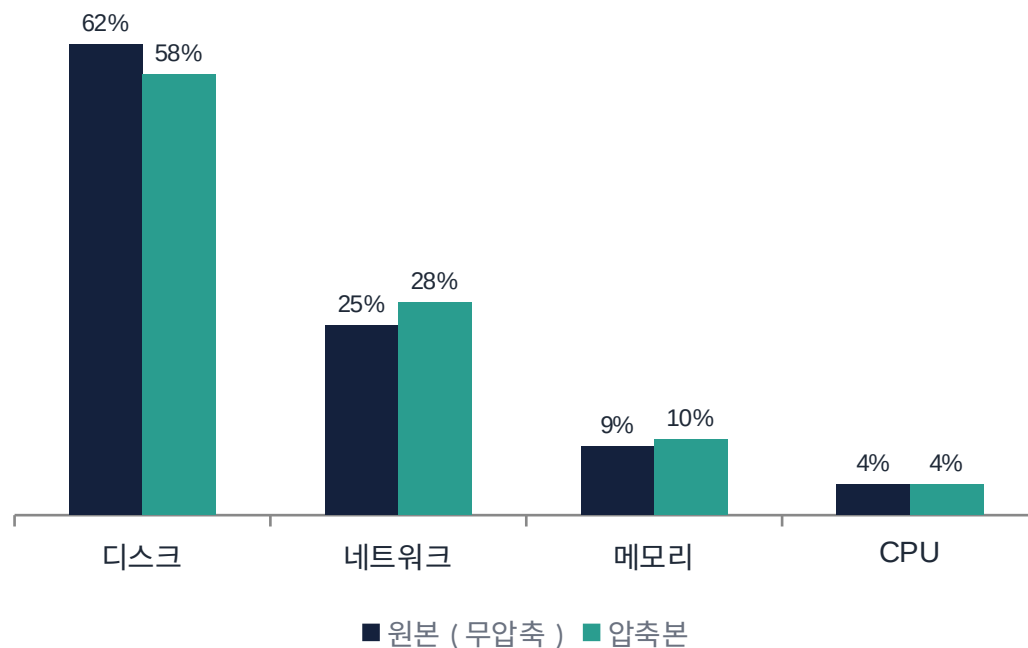
모델이 답 후보마다 매긴 확률의 묶음 . 제일 높은 걸 고른 게 '정답' .

답이 같다 ≠ 분포가 같다

정답률은 프론티어도 3.9~12.5% 로 낮아 압축 차이가 안 보인다 .

→ ' 맞았나 ' 대신 ' 확률 지형이 같나 ' 로 예민하게 본다 . 정보를 잃으면 확률이 뭉개진다 .

원본 vs 압축본 출력 분포 — 지형이 비슷하면 정보 보존 ✓



3 층 세 지표가 무엇을 하나

1

log-probability

정답 토큰에 준 확률 (로그)

0 에 가까울수록 정답을 확신 . 정답 ' 하나 ' 의 확신도를 본다 .

2

perplexity

$e^{(-\text{평균 log-prob})}$

log-prob 를 ' 체감 혼란도 ' 로 바꾼 값 . 낮을수록 안 헛갈림 = 좋음 .

3

KL divergence

$\sum P(i) \cdot \log(P(i)/Q(i))$

두 분포가 얼마나 벌어졌나 . 0 이면 같음 , 클수록 다름 . 분포 ' 전체 ' 를 본다 .

묶어 보면 log-prob·perplexity = ' 정답 하나의 확신' , KL = ' 분포 전체의 차이' . 둘을 함께 본다 .

다음 액션 — 2 층 평가 구체화

1.3 층 빠대는 정립 → 다음은 2 층 (효율 / 비용) 을 실제 측정 가능하게 못 박는다 .

1

측정 조건 고정

연산속도를 잘 GPU·배치·컨텍스트 길이를 통일 (공정 비교의 전제).

2

단위당 절감 확정

retained evidence 단위당 절감 계산식을 명확히 정의 .

3

2 층 검정력 채우기

필요 표본수 산정 — 1 층 AUC 검정력은 2 층에 못 쓰므로 별도 .

4

KV memory 측정법

GPU 피크 측정 vs 이론 계산 중 방식 확정 .